Du ska individuellt utveckla en liten men komplett **2D-plattformare i Unity**.

Du utgår från ett startrepository med grundläggande assets, scener och mappstruktur. All spelmekanik och de tekniska lösningar som examineras ska implementeras av dig.

Spelet ska innehålla:

- En sammanhängande spelbar bana som tar ungefär **45–90 sekunder** att spela.
- Spelarrörelse och hopp.
- Kollision med plattformar och miljö.
- Minst **en typ av fiende** med ett tydligt beteende.
- Minst **en annan fara** som kan påverka spelaren.
- Ett tydligt **vinstvillkor**.
- Ett tydligt **förlustvillkor**.
- Möjlighet att starta om efter vinst eller förlust.
- Tydlig återkoppling om spelets tillstånd.
- En komplett spelloop som kan spelas från start till slut utan att Unity-editorn behöver användas.

Projektet ska dessutom visa relevant användning av bland annat:

- Objektorienterad programmering.
- Datastrukturer.
- Events.
- Coroutines.
- Abstraktion och polymorfism.
- State Machine med minst två meningsfulla states.
- En styrande komponent för spelets övergripande flöde, exempelvis en GameManager.
- Unitys API och lifecycle på ett korrekt sätt.
- Strukturerad felsökning och felhantering.

Kvalitet före mängd. Fler banor, fiender eller mer grafik ger inte automatiskt ett högre betyg.

Betygskriterier från kursplan:

Examinationen bedömer främst din förmåga att:

- Felsöka och hantera fel strukturerat samt paketera och publicera en körbar version.
- Använda Git och Git LFS där det är relevant.
- Koppla ihop spelkomponenter med delegates, Actions, C# events eller Unity Events.
- Använda Unitys API och exekveringsmodell på ett korrekt och förutsägbart sätt.
- Använda coroutines där de fyller ett relevant behov.

- Välja och använda lämpliga datastrukturer.
- Implementera programlogik med variabler, operatorer, villkor och loopar.
- Använda objektorienterade principer såsom klasser, objekt, arv, polymorfism och abstraktion.

För **Väl godkänt** bedöms dessutom hur självständigt och metodiskt du arbetar, hur robust och underhållbar lösningen är samt hur väl tekniska val och designmönster är motiverade.

Betygsnivåer:

IG – Icke godkänt
Centrala krav saknas eller fungerar inte, projektet kan inte bedömas eller lösningen visar inte tillräcklig måluppfyllelse inom kursens lärandemål.

G – Godkänt
Spelet har en fungerande komplett spelloop med rörelse, fiende, fara, vinst, förlust och omstart. Unitys API, OOP, datastrukturer, events, coroutine och State Machine används på ett relevant och fungerande sätt. Projektet har en spårbar Git-historik och en fungerande, testad build publicerad på itch.io.

VG – Väl godkänt
Samtliga krav för G är uppfyllda. Lösningen håller dessutom en jämn, robust och professionell kvalitet. Ansvar och beroenden är tydligt strukturerade, tekniska val är välmotiverade och studenten visar en självständig och metodisk utvecklingsprocess med fokus på underhållbarhet och minskade sidoeffekter.

Instruktioner:

1. Ladda ner startprojektet och kontrollera att du använder rätt Unity-version.
2. Skapa ett eget Git-repository, bjud in läraren och pusha startprojektet innan du börjar utveckla.
3. Gör en första commit som visar att startprojektet går att öppna och köra.
4. Bygg först den minsta kompletta spelloopen: **start** → **spel** → **vinst/förlust** → **omstart**.
5. Implementera spelarrörelse, kollisioner och en spelbar bana innan du lägger tid på visuella detaljer.
6. Implementera fiendens State Machine med minst två meningsfulla states.
7. Använd datastruktur, events, coroutine och spelövergripande styrning där de fyller ett faktiskt behov.
8. Testa kontinuerligt både i Unity och i en fristående build.
9. Gör flera små och meningsfulla commits under arbetets gång.

10. Bygg projektet för **PC**.
11. Kontrollera att den publicerade versionen fungerar utanför Unity innan inlämning.

Arbetsform: Individuellt arbete.
Samarbete: Diskussion är inte tillåten, koddelning är inte tillåten.
Generativ AI: Inte tillåtet.
Inlämningsformat: Repository + körbar `.exe`.

Du ska lämna in:

- Hela Unity-projektet.
- Länk till Git-repository.
- Körbar version.
- `README.md` med kontroller, mål, vinst/förlust och bygginstruktioner.
- `TEKNISK_REFLEKTION.md`, cirka **500–900 ord**, där du beskriver kodstruktur, datastruktur, eventflöde, coroutine, State Machine, GameManager/Singleton eller alternativ lösning, felsökning och möjliga förbättringar.